

4.3 温度检测显示控制器的设计与制作

一、实验目的

- 1、学习和掌握温度检测显示控制器的基本原理；
- 2、掌握 Arduino 编程原理；
- 3、学习和掌握 PCB 电路板的设计和制作；
- 4、学习旋转编码器和 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 扩展板进行温度检测显示控制。硬件框图如图 4.13 所示，下面对各部分模块进行说明。

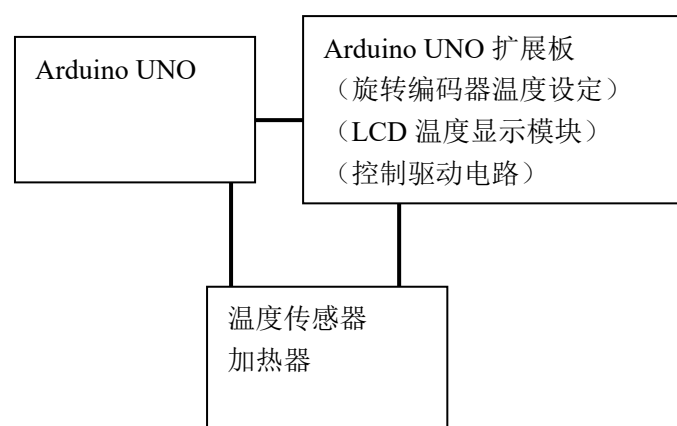


图 4.13 温度检测显示控制器框图

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字, LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准, 不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛, 最初的 1602 液晶使用的是 HD44780 控制器, 现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC, 所以特性上基本都是一致的。有关 LCD1602 模块的内容, 详见附录 1。

旋转编码器

旋转编码器是一种数字器件, 这种器件有两个输出 (A 和 B)。当旋转旋钮时, 输出会有变化, 这个变化就告诉你旋钮是顺时针旋转还是逆时针旋转。有关旋转编码器的内容, 详见附录 2。

温度传感器

LM35 是美国国家半导体公司生产的, 具有很高的工作精度和较宽的线性工作范围, 工作电压范围为 4~30V。其精度为 0.5°C , 输出电压与摄氏温度成线性比例关系, 非线性温度误差低于 $\pm 0.25^{\circ}\text{C}$, 线性温度系数为 $+10\text{mV}/^{\circ}\text{C}$, 无需外部校准或微调。输出电压与温度的表达式为: $U_{out} = 10\text{mV}/^{\circ}\text{C} \times t^{\circ}\text{C}$ 。


```

#include <LiquidCrystal.h>
//LiquidCrystal display with:
//rs on pin 12
//enable on pin 11
//d4-7 on pins 5-2
LiquidCrystal lcd(12,11,5,4,3,2);

int ledPin = A1;
int relayPin = A2;
int aPin = 8;
int bPin = 7;
int buttonPin = 6;
int analogPin = A0;

float setTemp = 20.0;
float measureTemp;
char mode = 'C'; //can be changed to F
boolean override = false;
float hysteresis = 0.25;

void setup() {
  lcd.begin(2,20);
  pinMode(ledPin,OUTPUT);
  pinMode(relayPin,OUTPUT);
  pinMode(aPin,INPUT);
  pinMode(bPin,INPUT);
  pinMode(buttonPin,INPUT);
  lcd.clear();
}

void loop() {
  static int count = 0;
  measureTemp = readTemp();
  if(!digitalRead(buttonPin))
  {
    override = !override;
    updateDisplay();
    delay(500); //debounce
  }
  int change = getEncoderTurn();

  setTemp = setTemp + change *0.1;

  if(count == 1000)

```

```

    {
        updateDisplay();
        updateOutputs();
        count = 0;
    }
    count++;
}

int getEncoderTurn()
{
    // return -1, 0, or +1
    static int oldA=HIGH;
    static int oldB=HIGH;
    int result = 0;
    int newA=digitalRead(aPin);
    int newB=digitalRead(bPin);
    if (newA != oldA || newB != oldB)
    {
        // something has changed
        if(oldA == HIGH && newA==LOW)
        {
            result = -(oldB*2-1);
        }
    }
    oldA = newA;
    oldB = newB;
    return result;
}

float readTemp()
{
    long a =analogRead(analogPin);
    float temp = 500.0*a/1023.0;
    return temp;
}

void updateOutputs()
{
    if(override||measureTemp<setTemp-hysteresis)
    {
        digitalWrite(ledPin,HIGH);
        digitalWrite(relayPin,HIGH);
    }
    else if(!override&&measureTemp>setTemp+hysteresis)
    {

```

```

        digitalWrite(ledPin, LOW);
        digitalWrite(relayPin, LOW);
    }
}
void updateDisplay()
{
    lcd.setCursor(0,0);
    lcd.print("Actual:");
    lcd.print(adjustUnits(measureTemp));
    lcd.print("o");
    lcd.print(mode);
    lcd.print(" ");

    lcd.setCursor(0,1);
    if(override)
    {
        lcd.print("OVERRIDE ON");
    }
    else
    {
        lcd.print("Set:");
        lcd.print(adjustUnits(setTemp));
        lcd.print("o");
        lcd.print(mode);
        lcd.print(" ");
    }
}
float adjustUnits(float temp)
{
    if(mode == 'C')
    {
        return temp;
    }
    else
    {
        return(temp*9)/5+32;
    }
}
}

```

控制系统中的迟滞问题考虑

设计这样一个温度控制器时需考虑到要防止“摆动”情况的出现。一个简单的通断控制系统，就会出现摆动情况。当温度降至设定值之下时，电源接通，开始加热，直至温度高于设定值，电源断开。然后慢慢变冷，直至温度再次低于设定值，此时加热器又被接通，如此

循环往复。但当温度恰好平衡在转变温度时，这种摆动会非常频繁，这种高频切换容易损坏器件。解决方法之一，就是引入迟滞（hysteresis）。图 4.17 展示如何利用一个迟滞值（迟滞变量设为 0.25°C ）避免高频摆动的方法。

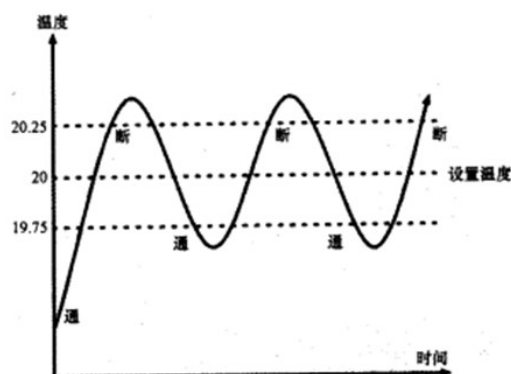


图 4.17 控制系统中的迟滞

温度随着电源接通而升高，它会到达设置点。然而，在温度超出设置点加上迟滞值之前电源并不会断开。同样，当温度下降时，降低到低于设置点时并不会重新加电，而是只有当温度下降到低于设置点减去迟滞值时才会加电。

显示闪烁问题考虑

编程时不要连续刷新显示值，因为读数的任何细微变化都可能导致显示乱闪烁。因此不要在每次主循环中刷新显示，而是循环多次（如 1000 次）刷新一次。可引入一个变量（如 counter），该变量每个循环增加 1 次，当其增加到 1000 时，刷新显示并将 counter 变量复位到 0。

每次改变显示值都使用 `lcd.clear()` 也会导致其闪烁。因此，可以采用将新温度值写在旧温度值的上面。

五、实验任务与要求

1、用 Arduino UNO 设计温度检测显示控制器，要求如下：

（1）选用合适的温度检测传感器，设计一个能实现测量范围 $0^{\circ}\text{C}\sim 100^{\circ}\text{C}$ ，测量精度为 0.5°C 的温度检测转换电路；

（2）设计一个温度加热控制电路，实现用旋转编码器设定控制温度值，当检测温度值低于设定值时，加热电路工作；当检测温度值高于设定值时，加热电路停止加热。

2、设计的电路，完成相应器件的选择和参数计算，制作扩展板和控制加热电路。

3、编制与调试温度检测显示控制器程序。

4、将制作的扩展板、控制加热电路板与 Arduino UNO 板组装后，进行系统联调。